

Cyber Threat Classification Under Non-Stationary Environments: An Adapt In-Context Learning (Adapt-ICL) Approach

Abstract

Pretrained large language models (LLMs) have demonstrated significant potential for cybersecurity analytics; however, evolving threat behaviors and shifting data distributions can undermine their reliability following deployment. Most existing adaptation methods require continual parameter updates, whereas retrieval-based in-context learning typically assumes that historical demonstrations remain representative over time. To overcome these challenges, Adapt-ICL is introduced as an inference-time adaptation framework that integrates Temporal Global Memory Evolution with Memory-Conditioned Hybrid Retrieval, while maintaining a frozen pretrained LLM backbone. Adapt-ICL is evaluated on the EMBER2024 malware benchmark using six general-purpose and cybersecurity-specific LLMs. Experimental results indicate that Adapt-ICL surpasses existing ICL-based approaches on Accuracy across all six LLMs and on Macro-F1 for five of the six, reduces the performance gap with continual fine-tuning, and sustains stable performance across sequential temporal intervals. These results underscore the effectiveness of memory-guided inference-time adaptation for robust cyber threat classification in non-stationary operational environments.

I. Introduction

Cybersecurity operations increasingly rely on machine learning (ML) and pretrained large language models (LLMs) to support a wide range of threat classification tasks, including malware detection, phishing detection, network intrusion detection, and prompt injection detection. These models have demonstrated strong representation learning capabilities and have become an important component of modern cybersecurity analytics (Ebrahimi et al., 2022; Li & Chai, 2022). However, unlike many conventional ML applications, cybersecurity environments are inherently non-stationary. Adversaries continuously evolve their tactics, techniques, and procedures (TTPs), while malware families adopt new obfuscation strategies and attack behaviors over time. Consequently, the statistical properties of operational data and the relationship between input features and class labels gradually change, causing the performance of deployed classifiers to deteriorate after deployment. Traditional adaptation strategies primarily rely on continual retraining or incremental parameter updates to maintain model performance. Although effective, these approaches are often computationally expensive and operationally difficult to deploy in real-world cybersecurity systems due to delayed security labels, high-throughput data streams, and the risk of catastrophic performance regression. As pretrained LLMs become increasingly adopted in cybersecurity applications, developing parameter-efficient adaptation methods that maintain reliable cybersecurity classification tasks performance without frequent backbone updates has therefore become an important research challenge.

Although pretrained LLMs have achieved significant success, sustaining their effectiveness post-deployment presents ongoing challenges. Current adaptation methods predominantly utilize continual fine-tuning or incremental parameter updates to integrate newly observed data. While these techniques can enhance predictive performance, they necessitate repeated model optimization and redeployment, resulting in substantial computational costs and maintenance difficulties, particularly in real-world cybersecurity contexts where timely labels are frequently unavailable (Parisi et al., 2019). Recently, retrieval-based in-context learning (ICL) has emerged as a promising paradigm for inference-time adaptation by dynamically selecting historical

demonstrations without altering model parameters (Dong et al., 2024). However, most retrieval strategies prioritize semantic similarity when ranking demonstrations and implicitly assume that the historical retrieval corpus remains representative over time. In cybersecurity environments, where malware characteristics, attack behaviors, and data distributions are in constant flux, semantically similar historical samples may not provide the most relevant evidence for current predictions. As a result, existing retrieval-based ICL methods are limited in their capacity to support reliable long-term cyber threat classification under non-stationary environments.

To address these limitations, we propose Adapt-ICL, an inference-time adaptation framework that organizes and retrieves historical cybersecurity knowledge without updating the pretrained LLM backbone. Instead of treating historical demonstrations as a static retrieval corpus, Adapt-ICL maintains a temporally evolving Global Memory that summarizes historical retrieval information and continuously updates it as new data becomes available. This evolving memory is subsequently incorporated into a memory-conditioned hybrid retrieval strategy to guide demonstration selection for each inference step. As a result, Adapt-ICL enables pretrained LLMs to adapt to evolving malware environments through retrieval optimization rather than parameter updates, providing a practical and computationally efficient alternative for long-term cyber threat detection.

We evaluate Adapt-ICL on the EMBER2024 malware benchmark with APK files only across six pretrained LLMs, including both general and cybersecurity LLMs. Experimental results demonstrate that Adapt-ICL outperforms existing ICL-based methods on Accuracy across all six LLMs and achieves performance competitive with parameter-updating baselines while maintaining a frozen pretrained LLM backbone. Furthermore, temporal evaluation shows that Adapt-ICL maintains stable detection performance under evolving malware distributions. These findings illustrate the potential of inference-time adaptation as a practical paradigm for maintaining reliable LLM-based cybersecurity tasks under evolving operational environments.

This study makes three contributions. First, we extend inference-time adaptation to non-stationary cybersecurity environments by proposing a memory-guided retrieval framework that keeps a frozen pretrained LLM backbone. Second, we introduce a Temporal Global Memory Evolution mechanism that continuously summarizes historical retrieval information and incorporates it into demonstration selection. This enables retrieval to adapt to evolving malware distributions. Third, through extensive experiments on the EMBER2024 benchmark across six pretrained LLMs, we provide empirical evidence that inference-time adaptation can substantially reduce the performance gap with continual fine-tuning while avoiding repeated parameter updates.

The remainder of this paper is organized as follows. Section II reviews the literature on cybersecurity analytics, inference-time adaptation, and retrieval-augmented in-context learning. Section III synthesizes the limitations identified in the literature and presents the research questions guiding this study. Section IV introduces the proposed Adapt-ICL framework and describes the research design and experimental setup. Section V reports the experimental results and discusses the key findings. Finally, Section VI concludes the paper and outlines directions for future research.

II. Literature Review

We review three streams of literature to establish the foundation of this research. First, we review studies on cybersecurity analytics to examine how computational artifacts have evolved from traditional machine learning approaches to pretrained LLMs for threat detection. Second, we review research on learning under non-stationary environments, focusing on how existing methods maintain model reliability after deployment and the limitations of parameter-updating paradigms. Finally, we review the literature on retrieval-augmented ICL to understand how inference-time

adaptation improves model performance through dynamic demonstration retrieval, while identifying the limited attention given to the temporal evolution of retrieved knowledge.

2.1 Cybersecurity Analytics in the Era of LLMs

As organizations undergo increasing digitization, cybersecurity analytics has become critical for identifying cyber threats, supporting Security Operations Centers (SOCs), and informing organizational security strategies. IS researchers have developed computational methods to detect cyber threats from diverse sources, including malicious websites, dark-web content, phishing communications, and other security data (Abbasi et al., 2010; Samtani et al., 2020; Ebrahimi et al., 2022). Recent advances in foundation models have further extended this line of research by enabling pretrained models to generalize across diverse downstream tasks (Bommasani et al., 2021). For instance, Ampel et al. (2026) introduced a large audio model for the automatic detection of voice phishing, illustrating the capacity of pretrained models to identify threat-related patterns in emerging data modalities. Concurrently, cybersecurity-specific LLMs such as Foundation-Sec-8B and RedSage have been developed through continued pre-training on security corpora and task-specific instruction tuning, enabling support for a broader range of cybersecurity analysis tasks (Kassianik et al., 2025; Suryanto et al., 2026).

In contrast to traditional supervised models that are designed for specific tasks, pretrained LLMs can transfer domain knowledge across multiple threat detection scenarios. This capability, however, introduces challenges in model maintenance. Cyber threats are continuously evolving as attackers adapt their TTPs, and as malware incorporates new obfuscation, command-and-control, and evasion strategies. As a result, knowledge derived from historical security data may become less applicable to newly emerging threats. Moreover, repeatedly fine-tuning and redeploying large pretrained LLMs is both computationally and operationally expensive, particularly when timely and reliable security labels are unavailable. Thus, a key challenge for LLM-enabled cybersecurity analytics is to maintain the post-deployment reliability of threat classifiers as operational data evolves, without relying on frequent full-model updates.

2.2 Learning under Non-Stationary Operational Environments

As machine learning models are increasingly deployed in business environments, researchers are paying more attention to their post-deployment reliability. Traditional supervised learning assumes that training and deployment data are drawn from the same distribution. However, operational environments frequently exhibit significant non-stationarity, where input features, labels proportions, and the relationships between features and labels may shift over time, resulting in performance degradation (Gama et al., 2014; Lu et al., 2019). To address these distribution shifts, research employs continual learning, incremental learning, or continual test-time adaptation to update model parameters, thereby leveraging newly arrived data and mitigating catastrophic forgetting (Wang et al., 2022; Niu et al., 2022). Nevertheless, recent studies indicate that prolonged online updates can result in error accumulation, prediction bias, and plasticity degradation, introducing new instabilities in dynamic environments (Chung et al., 2024; Shi et al., 2024).

While prior research has established a foundation for learning in non-stationary environments, most adaptation methods continue to depend on model parameter updates. This approach presents increasing challenges for pretrained LLMs, as continual fine-tuning and redeployment result in significant computational and operational costs (Kreuzberger et al., 2023). In the context of cybersecurity tasks, these challenges are exacerbated by delayed labels, which restrict the timely incorporation of newly observed data for parameter updates. Inference-time adaptation provides an alternative by modifying the information presented to a frozen model during

prediction. Nevertheless, current inference-time methods do not address how to maintain, summarize, or reweight the historical demonstration corpus as the underlying threat distribution changes (Ram et al., 2023; Long et al., 2023). In the absence of such mechanisms, retrieved demonstrations may remain tied to outdated or dominant historical patterns, thereby constraining post-deployment reliability in dynamic threat environments.

Recently, an increasing number of studies have begun exploring an adaptation paradigm that differs from traditional parameter updating. Inference-time adaptation offers an alternative to repeated full-model updating by adjusting the information used during inference while keeping the pretrained model backbone relatively stable. This paradigm is particularly relevant to dynamic operational environments, where models must respond to changing data distributions before sufficient labeled data become available for retraining. However, how inference-time adaptation can continuously maintain and utilize evolving historical knowledge is under-studied.

2.3 Retrieval-Augmented In-Context Learning

In-Context Learning is one of the most representative learning paradigms for inference-time adaptation in LLMs. Unlike traditional approaches that require parameter updates, ICL allows models to perform prediction tasks without retraining by supplying task instructions and a limited number of examples, referred to as "demonstrations," during the inference phase (Brown et al., 2020).

Consequently, significant research has examined the impact of demonstration selection on ICL performance. Findings indicate that ICL performance is determined not only by the number of demonstrations but also by their quality and the employed selection strategy (Liu et al., 2022; Rubin et al., 2022). To address these factors, certain approaches employ retrieval techniques to dynamically select demonstrations from historical samples that are most relevant to the current query, leading to the development of Retrieval-Augmented In-Context Learning. In comparison to random selection, retrieval-based ICL increases the relevance between demonstrations and the current task, thereby improving predictive performance and context utilization efficiency.

Subsequent research on retrieval-based ICL has expanded beyond simple semantic similarity to learned retrieval, iterative retrieval, and selection methods based on demonstration influence and gradient matching (Rubin et al., 2022; Ram et al., 2023; Long et al., 2023; Chen et al., 2024; Zhang et al., 2025a; Zhang et al., 2025b). These studies primarily focus on retrieving optimal demonstrations from a fixed candidate pool, with the goal of enhancing task performance by increasing semantic relevance, minimizing redundancy, or improving category coverage.

However, most retrieval-augmented ICL studies assume that the retrieval corpus remains static or changes slowly. In non-stationary environments, the relevance of historical demonstrations may change over time, while similarity-based retrieval may repeatedly favor common patterns and underrepresent rare or emerging threats. Therefore, effective long-term adaptation requires not only selecting relevant demonstrations, but also maintaining temporally representative and class-aware historical knowledge throughout deployment.

III. Current Limitations and Research Questions

We identified three important research limitations from our literature review. First, existing cybersecurity LLM studies primarily evaluate models under fixed train-test settings and do not provide a mechanism for maintaining reliable post-deployment threat detection as operational threat distributions evolve. Second, current approaches to non-stationary learning continue to

rely largely on repeated parameter updates, requiring continual model optimization and redeployment that are costly for large pretrained LLMs and difficult to sustain when timely security labels are unavailable. Third, existing retrieval-augmented ICL methods do not specify how historical demonstrations should be maintained and reweighted as their relevance changes over time, nor how a limited demonstration budget should preserve rare and emerging threats during retrieval. Based on these gaps, we pose the following research questions:

- **RQ1.** How can pretrained LLMs maintain reliable cyber threat detection under evolving operational environments without relying on frequent full-model updates?
- **RQ2.** How can inference-time adaptation continuously maintain and utilize evolving historical cybersecurity knowledge while keeping the pretrained LLM backbone stable?
- **RQ3.** How can retrieval-augmented ICL select temporally relevant demonstrations while preserving rare and emerging cyber threats under non-stationary environments?

IV. Proposed Research Design

Based on the identified research gaps, we propose **Adapt-ICL**, an inference-time adaptation framework for cyber threat detection under non-stationary environments (Figure 1). Our research design consists of three primary components: 1) Data Processing, 2) Adapt In-Context Learning (Adapt-ICL) Classifier, and 3) Experiments and Evaluation.

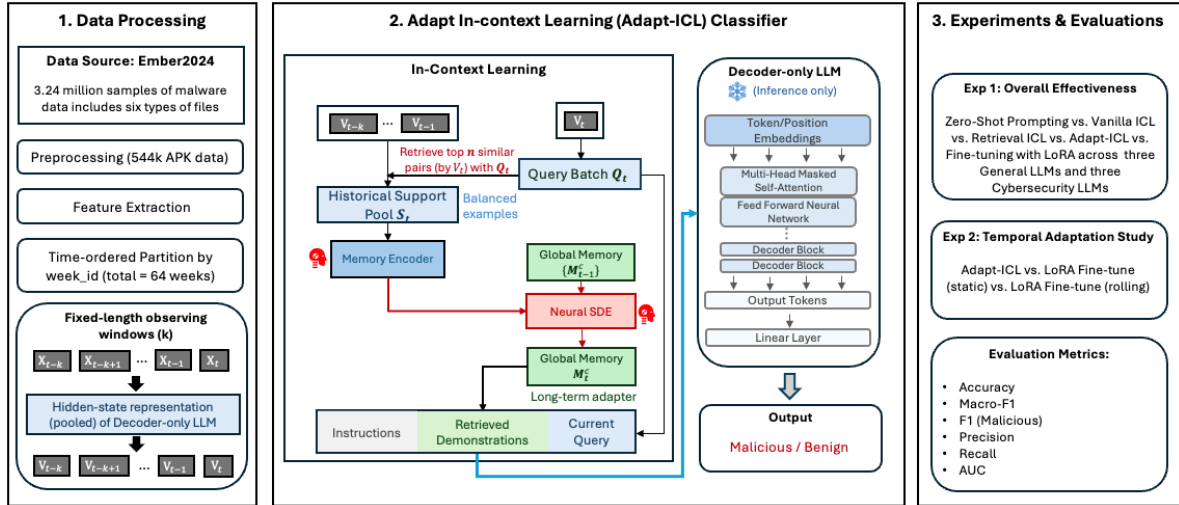


Figure 1: Proposed Research Design

4.1 Data Processing

EMBER2024 is a large-scale benchmark for the holistic evaluation of malware classifiers. It contains hashes, metadata, feature vectors, and labels for over 3.2 million files across six file formats (Win32, Win64, .NET, Android APK, Linux ELF, and PDF). It supports multiple malware classification tasks, including malware detection, family classification, and behavior identification (Joyce et al., 2025). As one of the most comprehensive publicly available malware benchmarks, EMBER2024 provides a unified platform for comparing malware classification methods across diverse malware ecosystems.

To align with the objective, we focus on the Android APK subset and formulate the task as a binary malware classification problem. Following the preprocessing pipeline in Figure 1, APK samples are extracted from the complete EMBER2024 benchmark and ordered by their temporal indices. Each sample is represented using the standardized static feature representation provided

by EMBER2024. At each time step t , observations from the previous W temporal intervals form the **Historical Support Pool** S_t . Observations arriving at time t form the **Current Query Batch** Q_t . This sliding-window organization transforms the original benchmark into a sequence of temporal inference tasks, enabling Adapt-ICL to retrieve recent historical support examples for evolving malware environments.

After preprocessing, the resulting dataset contains approximately **256,000** APK samples, including **208,000** training samples and **48,000** testing samples distributed across **64** consecutive temporal windows. These temporally ordered support-query pairs serve as the input to the proposed Adapt-ICL framework for all subsequent experiments.

4.2 Adapt In-Context Learning (Adapt-ICL) Classifier

In light of current limitations in threat detection, we propose Adapt-ICL, an inference-time adaptation framework for cyber threat detection in non-stationary operational environments. Unlike existing methods that require retraining or retrieval-augmented ICL approaches retrieving demonstrations from a relatively static historical support pool, Adapt-ICL continuously maintains and uses evolving historical cybersecurity knowledge to support reliable threat detection while keeping the pretrained LLM frozen during deployment. By avoiding frequent parameter updates to the LLM backbone, our framework enables efficient post-deployment adaptation and remains compatible with existing pretrained cybersecurity LLMs.

Our proposed Adapt-ICL framework has two primary components: 1) Temporal Global Memory Evolution continuously summarizes and updates historical cybersecurity knowledge using a Neural Stochastic Differential Equation (Neural SDE), and 2) Memory-Conditioned Hybrid Retrieval dynamically selects demonstrations based on semantic relevance, temporal recency, and class-aware representation.

4.2.1 Temporal Global Memory Evolution

Existing retrieval-based ICL methods retrieve demonstrations directly from a historical support pool based on their relevance to the current query. Although these approaches improve demonstration quality, they treat historical support samples as a static retrieval corpus. In non-stationary cybersecurity environments, malware characteristics and attack behaviors continuously evolve. Consequently, historical demonstrations may no longer accurately characterize the current threat landscape, reducing the effectiveness of inference-time adaptation.

To address this limitation, Adapt-ICL introduces a **Temporal Global Memory Evolution** module to maintain historical cybersecurity knowledge throughout deployment. As shown in Figure 1, at each time step t , a **Historical Support Pool** S_t is built from observations collected during the previous W temporal intervals. Instead of directly retrieving demonstrations from the support pool, Adapt-ICL first summarizes the current historical context into a **Local Memory** M_t . This is then integrated with the previous **Global Memory** M_{t-1}^c to generate an updated **Global Memory** M_t^c . This design maintains a recursively updated global state that integrates the sequence of Local Memories observed over time.

We define Historical Support Pool at time t as $S_t = \bigcup_{\tau=t-W}^{t-1} V_\tau$, where V_τ denotes the support samples collected during temporal interval τ , and W is the observation window size. After all queries at time t are processed, the demonstrations retrieved for these queries form a multiset D_t .

Each retrieved demonstration is represented by its backbone embedding x_i and one-hot class label e_{y_i} , and is transformed by the Memory Encoder: $z_i = f_{enc}(|x_i; e_{y_i}|)$, $i \in D_t$.

The encoded are first averaged within each represented class and then averaged across classes to construct the Local Memory: $M_t = \text{Aggregate}(\{z_i | x_i \in S_t\})$ where M_t summarizes the collective characteristics of the current historical support pool.

To continuously evolve historical knowledge across time, Adapt-ICL employs a Neural Stochastic Differential Equation (Neural SDE) to update the Global Memory. Neural SDEs model the evolution of a latent state through a drift function and a diffusion function (Li et al., 2020; Kidger et al., 2021). This formulation is suitable for non-stationary cybersecurity environments as the historical threat representation may exhibit both persistent distributional trends and irregular short-term fluctuations. In Adapt-ICL, the drift network propagates the Global Memory M_t^c as a function of the previous state, current Local Memory M_t , and time features, while the diffusion term introduces state-conditioned stochastic perturbations:

$$dM_t^c = f_\theta(M_{t-1}^c, M_t, \tau(t))dt + \sigma g_\phi(M_{t-1}^c, M_t, \tau(t))dW_t$$

$$\tau(t) = \left[\frac{t}{T}, \log(1+t) \right], \quad \sigma = 0.1$$

where $f_\theta(\cdot)$ models the deterministic evolution of historical knowledge, $g_\phi(\cdot)$ captures stochastic uncertainty introduced by newly emerging threats, W_t denotes a standard Wiener process, $\tau(t)$ denotes the time-feature vector, σ controls the diffusion scale, and the brackets denote vector concatenation.

4.2.2 Memory-Conditioned Hybrid Retrieval

At time t , Adapt-ICL uses the previously retained M_{t-1}^c to guide demonstration selection for the current Q_t . After all predictions at time t are completed, the retrieved demonstrations are used to construct M_t and update M_t^c , which is carried forward to the next temporal interval. The retrieval process considers three complementary factors. First, selected demonstrations should be relevant to the current query. Second, they should originate from recent temporal intervals. Third, the limited ICL context should maintain representation of both benign and malicious samples, preventing the majority class from dominating demonstration selection.

To achieve these objectives, Adapt-ICL proposes a Memory-Conditioned Hybrid Retrieval strategy. Instead of retrieving demonstrations solely according to semantic similarity, we incorporate the Global Memory into the query representation before demonstration retrieval. The memory-conditioned query is defined as

$$\tilde{x}_{q,t} = x_q + \alpha \|x_q\| \frac{M_{t-1}^c P}{\|M_{t-1}^c P\|}, \quad \alpha = 0.25$$

where x_q denotes the current query and P is a fixed random projection matrix that maps memory state into the query feature space. The term $\frac{M_{t-1}^c P}{\|M_{t-1}^c P\|}$ preserves only the direction of the projected

memory, while $\alpha \|x_q\|$ bounds the shift magnitude relative to the query norm. We set $\alpha = 0.25$. Then, the memory-conditioned query is used to retrieve demonstrations from the S_t . The highest-ranked candidates are subsequently filtered using temporal recency and class-aware constraints to construct the final demonstration set $D_t(q) = \text{HybridRetrieves}(S_t, \widetilde{x}_{q,t}, k)$, where k denotes the demonstration budget and $\text{HybridRetrieves}(\cdot)$ performs similarity ranking, temporal reweighting, and class-aware demonstration selection to construct the final retrieval set. The selected demonstrations are then combined with the task instruction and the current query to form the standard ICL prompt for the frozen pretrained LLM.

4.3 Experiments & Evaluation

We evaluated Adapt-ICL on the EMBER2024 malware benchmark using the Android APK subset introduced in Section 4.1. Each temporal interval contains approximately 4,000 APK samples. For each interval, up to 1,000 labeled support samples were selected according to the original class distribution to construct the Historical Support Pool. Throughout the experiments, the support window size was fixed to $W = 4$, so at most 4,000 historical support samples are available for each inference step. Query samples from the current interval were evaluated using only historical support data from previous intervals. No samples or labels from the current interval or any future interval were used for retrieval, memory evolution, or adaptation. Unless otherwise specified, the demonstration budget was fixed to $k = 6$.

We compared Adapt-ICL with Zero-shot, Vanilla ICL, retrieval-based ICL (Liu et al., 2022), and two Low-Rank Adaptation (LoRA; Hu et al., 2022) fine-tuning baselines across multiple pretrained LLMs. Specifically, LoRA-FT (Static) fine-tunes the model once using the initial training data and keeps the resulting parameters fixed throughout deployment, whereas **LoRA-FT (Rolling)** incrementally updates the model using newly available historical data at each temporal interval. These two settings represent offline and continual fine-tuning, respectively. We include them as stronger parameter-updating baselines to assess whether Adapt-ICL can achieve performance comparable to fine-tuning while keeping the pretrained LLM backbone frozen. Our evaluation consisted of two complementary aspects:

1. **Overall detection performance.** We compared Adapt-ICL with representative baseline methods across multiple pretrained LLMs using Accuracy, Macro-F1, and F1 on the malicious class (complete metrics appear in Appendix A3).
2. **Temporal adaptation study.** We evaluated model robustness by reporting performance across sequential temporal intervals and comparing the temporal stability between our proposed method and Fine-tuning.

V. Experiments and Results

5.1 Experiment 1: Overall Effectiveness

The results of Experiment 1 are shown in Table 1, with the best inference-time results in bold. All the metrics are the average of 12-week results. Adapt-ICL achieved the highest Accuracy among Zero-shot, Vanilla ICL, and retrieval-based ICL across all six LLMs (ACC+[0.75%-11.21%]) and the highest Macro-F1 on five of the six (Macro-F1+[0.75%-10.53%]); Lily-Cybersecurity was the only exception on Macro-F1. These results show that Adapt-ICL improves malware detection across different general-purpose and cybersecurity-specific LLMs.

Adapt-ICL also performed competitively against the parameter-updating baselines. It outperformed LoRA-FT (Static) on Llama-3.1, Foundation-Sec-8B, RedSage, and Mistral. Accuracy improvements across these four models ranged from 5.62% to 12.33%, while Macro-F1 improved from 5.29% to 18.38%. Although LoRA-FT (Rolling) achieved the strongest overall results, it requires repeated parameter updates with newly available historical data. Adapt-ICL's performance indicates that maintaining and retrieving evolving historical knowledge can partially mitigate temporal distribution shifts without updating the pretrained LLM backbone.

Retrieval-based ICL produced only limited changes compared with Vanilla ICL. It improved Accuracy on Llama-3.1 and Mistral but reduced Macro-F1 on five of the six LLMs and malicious-class F1 on five of the six. These results indicate that retrieving demonstrations based mainly on semantic similarity did not consistently improve model inference in this setting.

	Llama-3.1			Foundation-Sec-8B		
Method	Accuracy	Macro-F1	F1 (Malicious)	Accuracy	Macro-F1	F1 (Malicious)
Zero-shot	0.518	0.510	0.504	0.563	0.549	0.575
Vanilla ICL (k=6)	0.609	0.592	0.516	0.593	0.587	0.545
Retrieval ICL	0.612	0.589	0.496	0.583	0.572	0.511
Adapt-ICL	0.713	0.694	0.623	0.705	0.692	0.635
LoRA-FT (Static)	0.589	0.569	0.479	0.640	0.635	0.603
LoRA-FT (Rolling)	0.741	0.738	0.720	0.746	0.743	0.735
	Qwen3			RedSage		
Method	Accuracy	Macro-F1	F1 (Malicious)	Accuracy	Macro-F1	F1 (Malicious)
Zero-shot	0.552	0.547	0.529	0.550	0.545	0.529
Vanilla ICL (k=6)	0.596	0.585	0.533	0.620	0.609	0.556
Retrieval ICL	0.587	0.570	0.490	0.603	0.594	0.549
Adapt-ICL	0.623	0.592	0.489	0.699	0.693	0.673
LoRA-FT (Static)	0.642	0.639	0.621	0.643	0.640	0.627
LoRA-FT (Rolling)	0.741	0.738	0.732	0.774	0.773	0.761
	Mistral			Lily-Cybersecurity		
Method	Accuracy	Macro-F1	F1 (Malicious)	Accuracy	Macro-F1	F1 (Malicious)
Zero-shot	0.489	0.418	0.569	0.557	0.548	0.493
Vanilla ICL (k=6)	0.592	0.584	0.545	0.590	0.580	0.527
Retrieval ICL	0.601	0.594	0.547	0.527	0.507	0.419
Adapt-ICL	0.692	0.677	0.610	0.598	0.567	0.456
LoRA-FT (Static)	0.577	0.493	0.287	0.685	0.684	0.680
LoRA-FT (Rolling)	0.720	0.710	0.682	0.749	0.747	0.747

Table 1: Experimental results for Adapt-ICL and baseline methods in malware detection on EMBER2024 (APK-only) using six LLMs¹.

5.2 Temporal Adaptation Analysis

To further evaluate the robustness of Adapt-ICL under non-stationary operational environments, we compared its temporal performance with LoRA-FT (Static) and LoRA-FT (Rolling). Figure 2 illustrates the Macro-F1 of the three methods over twelve consecutive temporal intervals using Foundation-Sec-8B.

¹ Three architecture-matched model pairs were evaluated: Llama-3.1-8B/Foundation-Sec-8B, Qwen3-8B/RedSage-Qwen3-8B, and Mistral-7B/Lily-Cybersecurity-7B. Model details and complete results are provided in Appendices A3 and A4.

As shown in Figure 2, Adapt-ICL maintains a substantially more stable performance than LoRA-FT (Static) throughout the evaluation period. While the performance of LoRA-FT (Static) fluctuates considerably across different temporal intervals (Std = 0.0431), Adapt-ICL exhibits much smaller performance variations (Std = 0.0295), indicating that the proposed Temporal Global Memory Evolution effectively preserves useful historical knowledge while continuously adapting retrieval to evolving malware distributions.

Compared with LoRA-FT (Rolling), Adapt-ICL achieves slightly lower average performance but exhibits a comparable level of temporal stability. The standard deviations of the two methods are 0.0295 and 0.0255, suggesting that Adapt-ICL is able to maintain consistent detection performance without requiring continual parameter updates. These results further demonstrate that inference-time adaptation through memory-guided retrieval provides an effective alternative for maintaining model reliability under evolving operational environments.

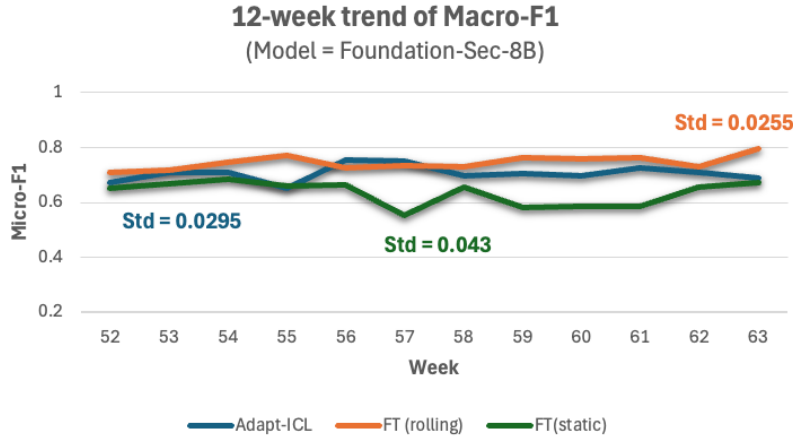


Figure 2: Macro-F1 of Adapt-ICL, LoRA-FT (Static), and LoRA-FT (Rolling) from weeks 52 to 63 on Foundation-Sec-8B (EMBER2024, APK-only).

VI. Conclusion and Future Directions

Reliable cyber threat detection in non-stationary operational environments remains a challenge for pretrained LLMs. Existing adaptation approaches mainly update model parameters through continual or rolling fine-tuning, which is computationally and operationally costly for large models and depends on timely security labels. Retrieval-augmented ICL offers a parameter-efficient alternative, but current methods generally select demonstrations from relatively static historical corpora whose relevance may decline as threat distributions evolve.

We introduced Adapt-ICL, an inference-time adaptation framework that integrates Temporal Global Memory Evolution with Memory-Conditioned Hybrid Retrieval while keeping the pretrained LLM backbone frozen. On the APK subset of EMBER2024, Adapt-ICL achieved higher accuracy than Zero-shot, Vanilla ICL, and retrieval-based ICL across six pretrained LLMs. It performed competitively against parameter-updating baselines and showed temporal stability comparable to rolling LoRA fine-tuning. These findings show that maintaining and conditioning retrieval on evolving historical knowledge can improve post-deployment threat detection without repeated backbone updates. Future work will strengthen the retrieval mechanism, incorporate soft-prompt adaptation for multi-class and multi-label tasks, and evaluate Adapt-ICL on prompt injection, network intrusion, and phishing detection.

References

1. Abbasi, A., Zhang, Z., Zimbra, D., Chen, H. and Nunamaker, J.F., 2010. Detecting fake websites: The contribution of statistical learning theory. *MIS Quarterly*, 34(3), pp.435–461.
2. Ampel, B.M., Samtani, S. and Chen, H., 2026. Automatically detecting voice phishing: A large audio model approach. *MIS Quarterly*, 50(2), pp.527–556.
3. Bommasani, R., Hudson, D.A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M.S., Bohg, J., Bosselut, A., Brunskill, E., et al., 2021. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*.
4. Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al., 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, pp.1877–1901.
5. Chen, Y., Chen, T., Jhamtani, H., Xia, P., Shin, R., Eisner, J. and Van Durme, B., 2024. Learning to retrieve iteratively for in-context learning. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
6. Chung, I., Hwang, K., Yoo, J., & Kwak, N. (2025). Mitigating the bias in the model for continual test-time adaptation. *IEEE Access*, 13, 150473–150488.
7. Dong, Q., Li, L., Dai, D., Zheng, C., Ma, J., Li, R., Xia, H., Xu, J., Wu, Z., Chang, B., Sun, X., Li, L. and Sui, Z., 2024. A survey on in-context learning. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp.1107–1128.
8. Ebrahimi, M., Chai, Y., Samtani, S. and Chen, H., 2022. Cross-lingual cybersecurity analytics in the international dark web with adversarial deep representation learning. *MIS Quarterly*, 46(2), pp.1209–1226.
9. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M. and Bouchachia, A., 2014. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), Article 44.
10. Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L. and Chen, W., 2022. LoRA: Low-rank adaptation of large language models. *Proceedings of the International Conference on Learning Representations (ICLR 2022)*.
11. Joyce, R.J., Miller, G., Roth, P., Zak, R., Zaresky-Williams, E., Anderson, H.S., Raff, E. and Holt, J., 2025. EMBER2024: A benchmark dataset for holistic evaluation of malware classifiers. *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD 2025)*.
12. Kassianik, P., Saglam, B., Chen, A., Nelson, B., Vellore, A., Aufiero, M., Burch, F., Kedia, D., Zohary, A., Weerawardhena, S., Priyanshu, A., Swanda, A., Chang, A., Anderson, H., Oshiba, K., Santos, O., Singer, Y. and Karbasi, A., 2025. Llama-3.1-FoundationAI-SecurityLLM-Base-8B technical report. *arXiv preprint arXiv:2504.21039*.
13. Kidger, P., Foster, J., Li, X. and Lyons, T., 2021. Efficient and accurate gradients for neural SDEs. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34, pp. 18747–18761.
14. Kreuzberger, D., Kühn, N. and Hirschl, S., 2023. Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11, pp.31866–31879.
15. Li, W. and Chai, Y., 2022. Assessing and enhancing adversarial robustness of predictive analytics: An empirically tested design framework. *Journal of Management Information Systems*, 39(2), pp.542–572.

16. Li, X., Wong, T.-K.L., Chen, R.T.Q. and Duvenaud, D., 2020. Scalable gradients for stochastic differential equations. *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS)*, PMLR 108, pp.3870–3882.
17. Liu, J., Xu, Y., Xu, H., Liu, Y., Cao, Y., Dou, Z. and Wen, J.R., 2022. What makes good in-context examples for GPT-3? *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures*, at ACL 2022, pp.100–114.
18. Long, Q., Wang, W. and Pan, S., 2023. Adapt in contexts: Retrieval-augmented domain adaptation via in-context learning. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
19. Lu, J., Liu, A., Dong, F., Gu, F., Gama, J. and Zhang, G., 2019. Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12), pp.2346–2363.
20. Niu, S., Wu, J., Zhang, Y., Chen, Y., Zheng, S., Zhao, P. and Tan, M., 2022. Efficient test-time model adaptation without forgetting. *Proceedings of the 39th International Conference on Machine Learning (ICML)*, PMLR 162, pp.16888–16905.
21. Parisi, G.I., Kemker, R., Part, J.L., Kanan, C. and Wermter, S., 2019. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113, pp.54–71.
22. Ram, O., Levine, Y., Dalmedigos, I., Muhlgay, D., Shashua, A., Leyton-Brown, K. and Shoham, Y., 2023. In-context retrieval-augmented language models. *Transactions of the Association for Computational Linguistics*, 11, pp.1316–1331.
23. Rubin, O., Herzig, J. and Berant, J., 2022. Learning to retrieve prompts for in-context learning. *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pp.2655–2671.
24. Samtani, S., Zhu, H. and Chen, H., 2020. Proactively identifying emerging hacker threats from the dark web: a diachronic graph embedding framework (D-GEF). *ACM Transactions on Privacy and Security*, 23(4), pp.1 to 33.
25. Shi, Z., Lyu, F., Liu, Y., Shang, F., Hu, F., Feng, W., Zhang, Z. and Wang, L., 2024. Controllable continual test-time adaptation. *arXiv preprint arXiv:2405.14602*.
26. Suryanto, N., Naseer, M., Li, P., Wasim, S.T., Yi, J., Gall, J., Ceravolo, P. and Damiani, E., 2026. RedSage: A cybersecurity generalist LLM. *Proceedings of the International Conference on Learning Representations (ICLR 2026)*.
27. Wang, Q., Fink, O., Van Gool, L. and Dai, D., 2022. Continual test-time domain adaptation. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
28. Zhang, J., Li, B., Bai, J., Li, R., Wang, Y., Lin, C. and Rong, W., 2025a. Selecting demonstrations for many-shot in-context learning via gradient matching. *Findings of the Association for Computational Linguistics: ACL 2025*, pp.11686–11704.
29. Zhang, Z., Zhang, Z., Li, D., Wang, L., Dy, J. and Zhang, H.R., 2025b. Linear-time demonstration selection for in-context learning via gradient estimation. *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp.16459–16477.

Appendix

A1. Operations of Adapt-ICL

We summarize the sequential operation of Adapt-ICL in Algorithm 1. At each time step t , the framework constructs a Historical Support Pool S_t from the most recent W temporal windows. The Global Memory retained from the previous time step, M_{t-1}^c , conditions each query representation and guides class-aware demonstration retrieval. The selected demonstrations are then combined with the current query and task instruction for prediction by the frozen pretrained LLM, and the demonstrations retrieved across all queries form the multiset D_t . After all queries at time t are processed, D_t is encoded into the Local Memory M_t , which is used to update the Global Memory M_t^c for retrieval at the next time step. The Memory Encoder is updated online using only the labels of historical demonstrations in D_t ; labels from the current query batch are used solely for evaluation and are never included in retrieval, memory evolution, or adaptation.

Algorithm 1 Adapt-ICL

Require: Temporal data $\{V_t\}_{t=1}^T$, window size W , demonstration budget k , frozen LLM F_{LLM}
Ensure: Predictions $\{\hat{y}_{q,t}\}$

- 1: Initialize Global Memory $M_0^c \leftarrow \mathbf{0}$
- 2: **for** $t = 1$ to T **do**
- 3: Construct Historical Support Pool S_t from the last W available intervals
- 4: Obtain Current Query Batch Q_t
- 5: Initialize retrieved-demonstration multiset $D_t \leftarrow \emptyset$
- 6: **for all** $q \in Q_t$ **do**
- 7: Condition query using M_{t-1}^c
- 8: Retrieve k class-aware demonstrations $D_t(q)$ from S_t
- 9: Predict $\hat{y}_{q,t} \leftarrow F_{\text{LLM}}(I, D_t(q), q)$
- 10: $D_t \leftarrow D_t \uplus D_t(q)$
- 11: **end for**
- 12: Encode D_t to obtain Local Memory M_t
- 13: Update Global Memory M_t^c using one Neural SDE step
- 14: **end for**
- 15: **return** $\{\hat{y}_{q,t}\}$

Algorithm 1: Operations of Adapt-ICL

The algorithm maintains a one-step temporal separation between adaptation and prediction. All queries at time t are processed using M_{t-1}^c , while the updated memory M_t^c is used only from time $t + 1$ onward. This design prevents temporal leakage and enables Adapt-ICL to adjust demonstration selection over time without updating the pretrained LLM backbone.

A2. Data Summary and Description

	Train	Test	Challenge
Time span	2023-09-24 – 2024-09-21	2024-09-22 – 2024-12-14	Full span
Weeks (time steps)	52 (weeks 0–51)	12 (weeks 52–63)	64
Samples	5,252,000	1,212,000	6,315
Samples per week	101,000	101,000	Varies
Malicious rate	50%	50%	100% (Malicious-only)

Table 3: EMBER2024 split summary.

File type	Rows per week	Share	Malicious rate
Win32	60,000	59.4%	0.500
Win64	20,000	19.8%	0.500
.NET	10,000	9.9%	0.500
APK	8,000	7.9%	0.500
PDF	2,000	2.0%	0.500
ELF	1,000	1.0%	0.500
Total	101,000	100%	0.500

Table 4: Per-week file-type composition in EMBER2024.

Component	Per interval	Total / operational scope
Support samples	Up to 1,000 (uniform, seeded)	Up to 4,000 per inference step (4-interval window)
Evaluation queries	Up to 2000 (uniform, seeded)	2,4000 across 12 intervals
LoRA-FT training subset	600 sampled from the support window	7,200 row-updates for rolling fine-tuning
Threshold calibration	64 support samples	Performed at each inference step

Table 5: Sampling protocol used in the 12-week evaluation.

EMBER2024 (Joyce et al., 2025) is a temporally organized malware-detection corpus spanning 64 consecutive weeks from September 24, 2023, to December 14, 2024. It contains six file formats (Win32, Win64, .NET, APK, PDF, and ELF) and is stratified by week, file type, and binary label. The first 52 weeks form the training split, the final 12 weeks form the temporally disjoint test split, and the malicious-only challenge split is excluded from the binary evaluation. The present study uses the APK subset. At each inference step, up to 1,000 labeled support samples are drawn from each of the preceding four available intervals, and up to 2000 samples from the current interval form the evaluation queries. The observed query balance was 51.1% benign and 48.9% malicious.

A3. Detailed Information of LLMs

Model	Hugging Face checkpoint	Parameters
Llama-3.1-8B-Instruct	meta-llama/Llama-3.1-8B-Instruct	8B
Foundation-Sec-8B-Instruct	fdtn-ai/Foundation-Sec-8B-Instruct	8B
Qwen3-8B	Qwen/Qwen3-8B	8B
RedSage-Qwen3-8B-Ins	RI Sys-Lab/RedSage-Qwen3-8B-Ins	8B
Mistral-7B-Instruct-v0.3	mistralai/Mistral-7B-Instruct-v0.3	7B
Lily-Cybersecurity-7B-v0.2	segolilylabs/Lily-Cybersecurity-7B-v0.2	7B

Table 4: LLM checkpoints used in the experiments.

The six checkpoints form three approximately size- and architecture-matched pairs: Llama-3.1-8B/Foundation-Sec-8B, Qwen3-8B/RedSage-Qwen3-8B, and Mistral-7B/Lily-Cybersecurity-7B. Within each pair, a general-purpose model is compared with a cybersecurity-specialized model, which is continue pretrained on the matched general LLM with cybersecurity domain corpus.

A4. Detailed E1 Results

Model	Method	Accuracy	Precision	Recall	Macro-F1	F1 (Malicious)	AUC	Query Time (s)
Llama-3.1	Zero-shot	0.5175	0.5073	0.5164	0.5101	0.5043	0.5221	1.0735
	Vanilla ICL	0.6087	0.6496	0.4353	0.5921	0.5155	0.6628	4.6003
	Retrieval ICL	0.6117	0.6887	0.3983	0.5889	0.4959	0.6915	4.6036
	Adapt-ICL	0.7125	0.86	0.4995	0.6937	0.6226	0.8012	1.0233
	LoRA-FT (Static)	0.5892	0.6367	0.3855	0.5692	0.4791	0.6655	1.1631
	LoRA-FT (Rolling)	0.7413	0.7621	0.6921	0.7377	0.7196	0.8258	1.1625
Qwen3	Zero-shot	0.5521	0.5462	0.5259	0.5474	0.529	0.5674	1.1878
	Vanilla ICL (k=6)	0.5963	0.6094	0.4846	0.5847	0.5326	0.624	5.9319
	Retrieval ICL	0.5871	0.6249	0.4144	0.5697	0.49	0.643	5.992
	Adapt-ICL	0.6225	0.7033	0.389	0.5922	0.489	0.7094	1.4545
	LoRA-FT (Static)	0.6417	0.6468	0.6023	0.6389	0.6214	0.7131	1.1966
	LoRA-FT (Rolling)	0.7408	0.7408	0.7307	0.7384	0.7322	0.8364	1.1962
Mistral	Zero-shot	0.4887	0.4848	0.736	0.4183	0.569	0.5251	1.2512
	Vanilla ICL	0.5917	0.5965	0.5084	0.584	0.5449	0.6186	6.5307
	Retrieval ICL	0.6008	0.6197	0.5014	0.5935	0.5473	0.6585	6.6273
	Adapt-ICL	0.6917	0.8003	0.499	0.6768	0.6102	0.7803	1.5649
	LoRA-FT (Static)	0.5771	0.8157	0.1751	0.493	0.2871	0.7077	1.1791
	LoRA-FT (Rolling)	0.7196	0.7613	0.6576	0.7097	0.6815	0.8068	1.1864

Table : Complete experimental results of all baseline methods and Adapt-ICL across pretrained general LLMs.

Model	Method	Accuracy	Precision	Recall	Macro-F1	F1 (Malicious)	AUC	Query Time (s)
Foundation-Sec-8B	Zero-shot	0.5625	0.5426	0.6299	0.5493	0.5751	0.5752	1.0725
	Vanilla ICL	0.5933	0.5973	0.5052	0.5868	0.5452	0.6388	4.6127
	Retrieval ICL	0.5833	0.5977	0.4538	0.5719	0.5106	0.6189	4.6466
	Adapt-ICL	0.7054	0.8118	0.5406	0.6921	0.635	0.7686	1.0222
	LoRA-FT (Static)	0.6396	0.6583	0.5605	0.6348	0.6026	0.7326	1.1633
	LoRA-FT (Rolling)	0.7462	0.7542	0.7277	0.7426	0.7345	0.8329	1.1628
Lily-Cybersecurity	Zero-shot	0.5571	0.566	0.4458	0.548	0.4926	0.5828	0.837
	Vanilla ICL	0.59	0.602	0.4728	0.5799	0.5265	0.6084	5.0552
	Retrieval ICL	0.5271	0.5374	0.36	0.5071	0.4193	0.5684	5.1577
	Adapt-ICL	0.5975	0.683	0.3463	0.5665	0.4556	0.6427	1.6151
	LoRA-FT (Static)	0.685	0.6792	0.6846	0.684	0.6804	0.7711	1.1874
	LoRA-FT (Rolling)	0.7492	0.7472	0.7561	0.747	0.7467	0.837	1.1886
RedSage	Zero-shot	0.55	0.5401	0.526	0.5454	0.5292	0.5585	1.1933
	Vanilla ICL	0.62	0.6497	0.4984	0.6089	0.5562	0.6634	5.9332
	Retrieval ICL	0.6025	0.6233	0.5094	0.5941	0.5494	0.6717	5.9831
	Adapt-ICL	0.6987	0.7276	0.6448	0.6932	0.673	0.7698	1.4461
	LoRA-FT (Static)	0.6425	0.643	0.6151	0.6403	0.6269	0.723	1.1967
	LoRA-FT (Rolling)	0.7742	0.7941	0.7339	0.7726	0.7609	0.8581	1.2136

Table 4: Complete experimental results of all baseline methods and Adapt-ICL across Cybersecurity LLMs.